```
  def eventHandler(state: State, event: Event): State =          ❻
    event match {
      case CargoAdded(containerId, cargo) =>
        state.copy(cargos = cargo +: state.cargos)
    }
}
```

❶ Messages to interact with the actor
❷ Event that can be triggered
❸ State of the actor
❹ Initializes the persistence entity
❺ Handles commands and triggers events
❻ Handles events triggered from commandHandler

`PersistenceId.ofUniqueId` checks that the value passed is not `null` or empty and re-creates `toString`, `hashCode`, and `equals`.

9.2.2 Persistence combined with sharding: A persistent entity

To guarantee the uniqueness of a persistent actor in a cluster, you can combine it with sharding. You need to add an `EntityTypeKey` and use it when setting the persis-tence ID of the persistent actor. To do so, you can use one of the `PersistenceId` constructors:

```
def apply(entityTypeHint: String, entityId: String)
```

Typically, you combine the `EntityTypeKey` with the actor's ID—here, `typeKey` with `containerId`:

```
object SPContainer {

  val typeKey =
    EntityTypeKey[SPContainer.Command]("spcontainer-type-key")      ❶

  ...

  def apply(containerId: String): Behavior[Command] =
    EventSourcedBehavior[Command, Event, State](
      PersistenceId(typeKey.name, containerId),                     ❶
      State(),
      commandHandler,
      eventHandler)

  ...
}
```

❶ Adds the EntityTypeKey of the entity
❷ Combines the containerId with the typeKey

To instantiate the actor, you need to treat it like an entity. Create a sharded cluster (an SR), and instantiate it from there: